TALLER PRÁCTICO INTEGRADOR

Estrategia de pruebas

Plataforma web de venta de boletos **CineFlex**

Fase A: Arquitectura y estrategia de aseguramiento de calidad

Pruebas de integración • Stub bancario • Pruebas de sistema • Regresión • Comparativa Selenium / Cypress

Modalidad: entorno de demostración (mock) con API REST, interfaz web y pasarela de pagos simulada.

Repositorio público
github.com/pollofrito18/qa-cineflex-project

Octubre de 2026

1. Contexto y alcance

CineFlex es una plataforma académica de venta de boletos de cine. Permite iniciar sesión, consultar la cartelera, seleccionar funciones y asientos, simular un pago y obtener un ticket. La evaluación se realiza sobre un entorno **mock**, no sobre un servicio bancario ni sobre infraestructura de producción.

1.1 Capas y responsabilidades

Capa	Responsabilidad en CineFlex	Objeto de prueba
Frontend (HTML, CSS y JavaScript)	Interfaz de login, cartelera, selección de asientos, pago y ticket.	Comportamiento visible, navegación y flujos de extremo a extremo.
Backend (Node.js / Express)	Autenticación JWT, cartelera, disponibilidad de asientos, reservas y lógica de pago simulado.	Contratos HTTP, reglas de negocio, seguridad y tiempos de respuesta.
Persistencia y servicio bancario simulado	Reservas mantenidas en memoria durante la ejecución y respuesta bancaria controlada por un Stub.	Consistencia de asientos, confirmación/rechazo de pagos y aislamiento de terceros.

1.2 Objetivo de la estrategia

Demostrar con pruebas reproducibles que CineFlex ejecuta sus transacciones principales, rechaza entradas inválidas y mantiene la disponibilidad de asientos coherente; además, priorizar la regresión según riesgo y automatizar comprobaciones sobre API y UI.

1.3 Alcance y exclusiones

- Incluye autenticación JWT, cartelera, disponibilidad, pagos de prueba, generación de tickets y navegación.
- Incluye validación automatizada con Postman/Newman y Selenium/POM, más una carga local controlada con JMeter.
- Excluye certificación real de una pasarela de pagos, persistencia de base de datos productiva y garantías de escalabilidad en infraestructura externa.

> **Distinción de arquitectura**
>
> El enunciado describe una capa relacional como arquitectura objetivo. La versión de CineFlex implementada para el taller conserva reservas en memoria; el PDF documenta expresamente esa limitación.

2. Pruebas de integración

2.1 Elección: integración Top-Down

Se propone **Top-Down** porque la interfaz de pago y el endpoint de reservas ya están preparados y el servicio bancario externo aún no se encuentra disponible en el entorno de pruebas. Se comienza validando el flujo de usuario y el controlador de reservas; la dependencia bancaria se sustituye temporalmente por un **Stub** determinista.

Frente a Bottom-Up, que comienza integrando componentes de bajo nivel con ayuda de drivers, este enfoque entrega antes evidencia del recorrido crítico **interfaz → API de reservas → respuesta del banco simulado**, sin bloquear el desarrollo a la espera del proveedor externo.

2.2 Diseño del Stub bancario

Entrada de prueba	Respuesta simulada	Comportamiento esperado
PRUEBA_APROBADA	Pago autorizado (respuesta interna satisfactoria).	Crear reserva, ocupar asiento y emitir ticket con estado CONFIRMADO.
TARJETA_FALSA	Pago rechazado, con código PAGO_RECHAZADO.	Responder HTTP 402; no generar ticket ni confirmar la ocupación del asiento.
Asiento previamente ocupado	La validación de negocio detiene la compra.	Responder HTTP 409 antes de concluir otra reserva sobre el mismo asiento.

2.3 Propuesta de implementación

El Stub recibe datos de la transacción y devuelve una respuesta predecible. El controlador de reservas valida primero la película, el horario y el asiento; luego consulta el Stub y crea el ticket solo si el pago fue aprobado. Un ejemplo de pseudocódigo adaptado a CineFlex es:

```
function simularBanco(tarjeta) {
  if (tarjeta === 'PRUEBA_APROBADA') {
    return { autorizado: true };
  }
  return { autorizado: false, codigo: 'PAGO_RECHAZADO' };
}
```

Nota: código ilustrativo de la estrategia. No es una transcripción literal de **server.js**; refleja los comportamientos verificados del entorno mock.

2.4 Casos mínimos de integración

- **IT-01:** pago de prueba aprobado → HTTP 201, ticket y asiento ocupado.
- **IT-02:** tarjeta falsa → HTTP 402, mensaje de rechazo y ausencia de ticket.
- **IT-03:** segundo intento de reservar un asiento ya ocupado → HTTP 409.
- **IT-04:** consulta de asientos antes y después de la compra → cambio de disponibilidad consistente.

3. Requisitos y pruebas de sistema

Los requisitos se formulan como criterios verificables y están asociados a los escenarios implementados en el mock. Se diferencian los **requisitos** de la evidencia de la ejecución puntual.

3.1 Dos requisitos funcionales

Código	Requisito	Criterio de aceptación
RF-01	El usuario debe poder iniciar sesión con credenciales válidas y acceder a la cartelera mediante autenticación JWT.	POST /api/auth/login responde HTTP 200 y entrega un token; GET /api/cartelera autenticado responde HTTP 200.
RF-02	El usuario debe poder reservar un asiento libre mediante el pago simulado; un pago rechazado no debe confirmar la venta.	Pago aprobado: ticket confirmado y asiento ocupado. Tarjeta falsa: HTTP 402 y sin ticket. Asiento duplicado: HTTP 409.

3.2 Dos requisitos no funcionales

Código	Requisito	Criterio de aceptación
RNF-01	La consulta autenticada de cartelera debe responder de forma ágil.	El tiempo de respuesta de GET /api/cartelera debe ser **menor a 300 ms** en la prueba de Postman/Newman.
RNF-02	La consulta de disponibilidad debe mantenerse operativa durante una carga local escalonada.	Con **500 usuarios virtuales** y **15 segundos de ramp-up**, registrar porcentaje de error y throughput. Meta de esta validación del mock: 0 % de errores.

3.3 Diseño de pruebas de sistema

- **Flujo positivo:** login → cartelera → película → horario → asiento disponible → pago aprobado → ticket.
- **Flujo negativo:** compra con tarjeta falsa → rechazo y ausencia de ticket.
- **Integridad:** impedir la doble reserva de un asiento ocupado.
- **Seguridad:** bloquear acceso a los recursos protegidos sin JWT y validar el comportamiento del cierre de sesión.

> **Interpretación del rendimiento**
>
> El SLA menor de 300 ms aplica a la consulta de cartelera en el entorno evaluado. La carga de JMeter se realizó sobre disponibilidad de asientos; no debe confundirse con una certificación de producción.

4. Estrategia de regresión

4.1 Principio de selección por impacto y riesgo

No se ejecuta toda la batería de pruebas después de cualquier cambio menor. Primero se determina qué módulos, estilos, contratos y dependencias podrían verse afectados. Se priorizan los caminos críticos: autenticación, reserva, asignación de asiento y pago. La ejecución completa se reserva para cambios transversales, integraciones relevantes o entregas de mayor riesgo.

4.2 Caso solicitado: color de un botón secundario

Si un desarrollador cambia **solo el color de un botón secundario en el perfil**, no se justifica repetir automáticamente toda la suite de la pasarela de pagos. Se realizan pruebas visuales, de contraste/accesibilidad y de navegación local; además, una comprobación básica de interfaz si el estilo utiliza reglas CSS compartidas. Si el mismo CSS afecta el botón de pagar, se amplía la regresión a la reserva.

4.3 Matriz de selección

Cambio	Pruebas de regresión prioritarias	Motivo
Color de botón secundario de perfil	Inspección visual, contraste y navegación; smoke test de UI si comparte CSS.	Bajo riesgo para el contrato bancario; evitar costo de ejecución innecesario.
Reglas de login o JWT	Login correcto/incorrecto, rutas protegidas, bloqueo, cierre de sesión y navegación posterior.	Impacto directo en acceso y seguridad.
Lógica de reservas o asientos	Integridad 409, actualización de disponibilidad, pago aprobado/rechazado, E2E y carga dirigida.	Riesgo de sobreventa y pérdida de consistencia.
Contrato del banco simulado	Casos 201/402, mensajes, ausencia de ticket al rechazar y recorrido E2E.	Cambios pueden alterar la regla financiera central.
Dependencias generales o despliegue	Suite API Newman en CI + smoke UI; ampliar según resultados.	Posible efecto transversal entre capas.

4.4 Orden recomendado

- **Primer nivel:** pruebas unitarias/contrato de componentes afectados.
- **Segundo nivel:** casos críticos API (JWT, cartelera, pagos).
- **Tercer nivel:** Selenium E2E focalizado.
- **Cuarto nivel:** suite completa y JMeter cuando el cambio afecta rendimiento, concurrencia o una entrega importante.

5. Selenium WebDriver frente a Cypress

Las herramientas tienen enfoques distintos. La elección debe responder al tipo de riesgo que se quiere cubrir en CineFlex, no a una preferencia general.

Criterio	Selenium WebDriver	Cypress
Arquitectura de control	Automatiza navegadores mediante WebDriver; útil para flujos próximos al uso real del usuario.	Se integra estrechamente al entorno del navegador para pruebas de interfaz y depuración rápida.
Cobertura de navegadores	Adecuado para matrices de compatibilidad y ejecución remota con diferentes navegadores y controladores.	Adecuado para desarrollo y verificación ágil en los navegadores soportados por el proyecto.
Patrón y mantenimiento	Se adapta de forma natural al Page Object Model exigido en el taller.	Suele favorecer comandos reutilizables, fixtures e interceptación de red; el modelo Page Object no es obligatorio.
Red y servicios externos	Permite validar el flujo visible mientras un Stub sustituye la pasarela de pagos.	Facilita interceptar y simular respuestas HTTP desde el propio test para aislar Frontend.
Escenario CineFlex	Verificar login, selección de asiento, pago y ticket en una regresión E2E.	Desarrollar rápidamente pruebas visuales del pago con respuestas HTTP simuladas y depuración interactiva.

5.1 ¿Cuándo elegir Selenium en CineFlex?

Cuando la prioridad es **validar el recorrido completo como lo haría una persona usuaria**, comprobar integración real UI/API en Chrome y preparar escenarios de compatibilidad con más navegadores. Es la opción principal implementada porque el taller exige un conjunto modular de al menos 15 pruebas con POM.

5.2 ¿Cuándo elegir Cypress en CineFlex?

Durante el desarrollo intensivo de la interfaz: por ejemplo, si el equipo necesita probar en minutos las diferentes respuestas del banco sin levantar todo el servicio de pagos, puede interceptar la llamada HTTP y devolver respuestas de aprobación y rechazo controladas. Esto mejora la rapidez de depuración y la independencia de pruebas de UI.

> **Decisión técnica**
>
> Se selecciona Selenium WebDriver para la entrega evaluada; Cypress se recomienda como complemento potencial en pruebas rápidas de componentes y contratos frontend simulados. No se afirma que ambos se hayan implementado.

6. Evidencia y trazabilidad

Los siguientes datos proceden de las ejecuciones del proyecto realizadas durante el taller. Son resultados de un entorno local/de integración, no indicadores universales de producción.

Evidencia	Resultado comprobado	Ubicación esperada
API: Postman y Newman	3 peticiones; 16 aserciones aprobadas; 0 fallidas. Login 200, cartelera 200, tarjeta falsa 402.	api_tests/ y capturas en evidencias/
Integración continua	GitHub Actions ejecutó Newman tras push; 16 aserciones exitosas en el pipeline.	.github/workflows/qa-pipeline.yml
Selenium / POM	15 casos ejecutados, 15 aprobados y 0 fallidos.	ui_tests/test_suite.js y ui_tests/pages/
Carga JMeter	500 consultas de asientos, 1 login, 0 % errores. Promedio de asientos ~3 ms; throughput de asientos ~33,4 req/s.	performance/prueba_carga.jmx y Summary Report

6.1 Límites metodológicos

- Los 500 usuarios se iniciaron escalonadamente durante 15 segundos: no equivale a afirmar 500 solicitudes simultáneas en un instante.
- El simulador mantiene el estado de reservas en memoria y lo reinicia al apagar el Backend.
- Las observaciones de latencia corresponden al equipo y configuración utilizados; pueden variar en otra máquina.
- Un porcentaje de error igual a cero en esta ejecución no constituye una garantía de estabilidad futura ni de seguridad integral.

6.2 Puntos de control para completar la entrega

- Verificar que la suite Selenium incluya expresamente los escenarios obligatorios de bloqueo tras tres intentos, título «Cartelera» en DOM y cierre de sesión seguido de botón «Atrás».
- Comprobar el script de desarrollo de backend frente a lo pedido literalmente por el enunciado: se utilizó node --watch en lugar de nodemon por un problema con dependencias.
- Documentar el estado de alertas de seguridad en dependencias de desarrollo y no publicar credenciales ni tokens reales.
- Incorporar capturas legibles del Summary Report en README, junto al enlace de video de sustentación.

7. Cierre y recomendaciones

La estrategia propuesta permite avanzar sin la API del banco por medio de un Stub controlado y pruebas Top-Down. La separación entre validación de API, automatización de interfaz y carga local ayuda a detectar problemas de autorización, disponibilidad, pagos y tiempos de respuesta con diferentes niveles de costo.

Los resultados registrados son satisfactorios para este entorno de demostración: las 16 aserciones API y los 15 casos UI reportaron éxito; JMeter registró 500 consultas de asientos sin error. Antes de considerar terminada la entrega académica, deben cerrarse los casos específicos de Selenium que exige expresamente el enunciado y completarse los materiales de presentación.

7.1 Relación entre fases y entregables

Fase	Archivo o evidencia	Estado reportado
A	estrategia_pruebas.pdf	Este documento.
B	backend/server.js; colección y entorno Postman; Newman	Implementado y probado.
C	frontend/; ui_tests/pages/; ui_tests/test_suite.js	15 pruebas ejecutadas. Revisar escenarios obligatorios.
D	performance/prueba_carga.jmx; qa-pipeline.yml; Summary Report	Implementado, probado y publicado.
Entrega	README con despliegue, evidencias y enlace de video	Pendiente de edición final.

Referencias y fuentes de proyecto

1. Enunciado del curso: **“Taller Práctico Integrador: QA en Producción (CineFlex)”**, archivo Taller.html proporcionado en la actividad.

2. Repositorio del proyecto: **https://github.com/pollofrito18/qa-cineflex-project**.

3. Evidencia de consola y capturas generadas durante la ejecución de Postman/Newman, Selenium WebDriver, JMeter y GitHub Actions, octubre de 2026.

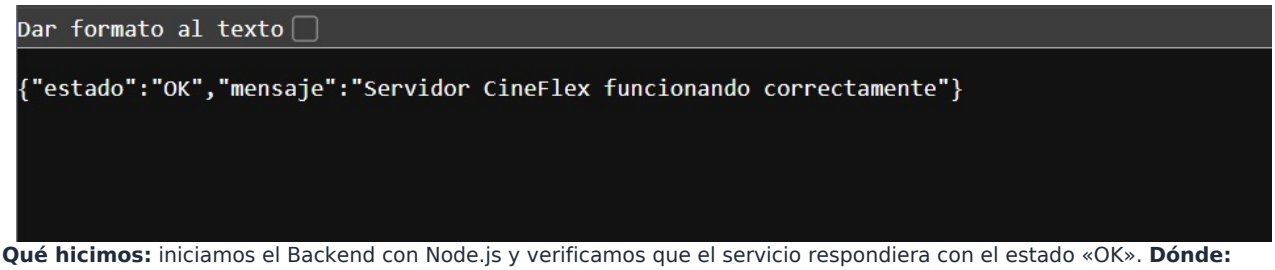
> **Alcance documental**
>
> Este archivo cubre la Fase A solicitada (integración, Stub, requisitos, regresión y comparativa de herramientas) y ofrece trazabilidad resumida hacia las fases B, C y D. Para la entrega, ubicarlo en la raíz de qa-cineflex-project/.

FIN DEL DOCUMENTO

ANEXO A. Evidencia visual y ubicación del trabajo

Las siguientes capturas proceden de las pruebas reales del entorno académico CineFlex. Cada figura indica **qué se hizo**, **dónde se hizo** y **qué se comprobó**. Son resultados de un entorno mock local, no de un despliegue bancario real.

A.1 Servidor API REST disponible



Qué hicimos: iniciamos el Backend con Node.js y verificamos que el servicio respondiera con el estado «OK». **Dónde:** backend/server.js, ejecutado con npm start; endpoint GET /api/health en localhost:3000. **Resultado:** el servidor respondió correctamente.

Figura A1. Respuesta JSON obtenida del servidor Express.

A.2 Selección de función y asiento



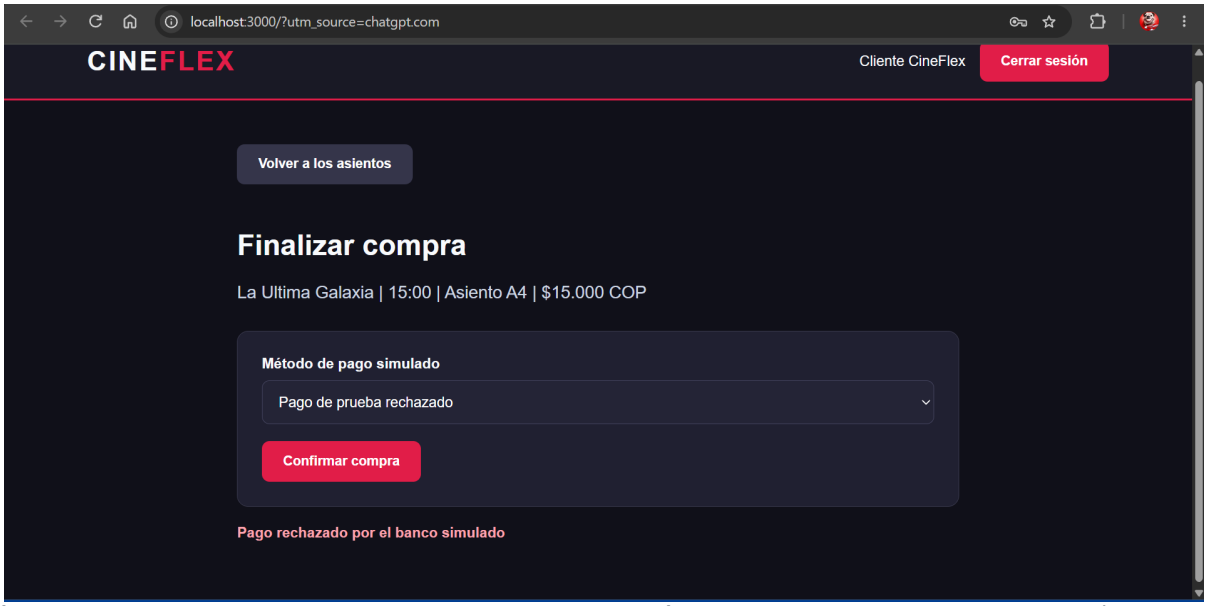
Qué hicimos: presentamos película, horario y asientos disponibles u ocupados. **Dónde:** interfaz en frontend/index.html, frontend/styles.css y frontend/app.js; datos desde GET /api/peliculas/:id/asientos. **Resultado:** la interfaz permite escoger un asiento libre y distingue los ocupados.

Figura A2. Pantalla de selección de asientos de la película La Última Galaxia.

ANEXO B. Integración con el banco simulado

Este anexo ilustra el enfoque Top-Down expuesto en la Fase A: la interfaz y la API se integran con una respuesta bancaria controlada, sin depender de un proveedor externo.

B.1 Caso negativo: tarjeta falsa



Qué hicimos: elegimos un medio de pago de prueba rechazado. **Dónde:** vista de pago en frontend/app.js y lógica de POST /api/reservas en backend/server.js. **Qué comprueba:** el Stub simula rechazo HTTP 402, se muestra un mensaje claro y no se emite ticket.

Figura B1. Mensaje «Pago rechazado por el banco simulado» en la interfaz.

B.2 Caso positivo: reserva confirmada

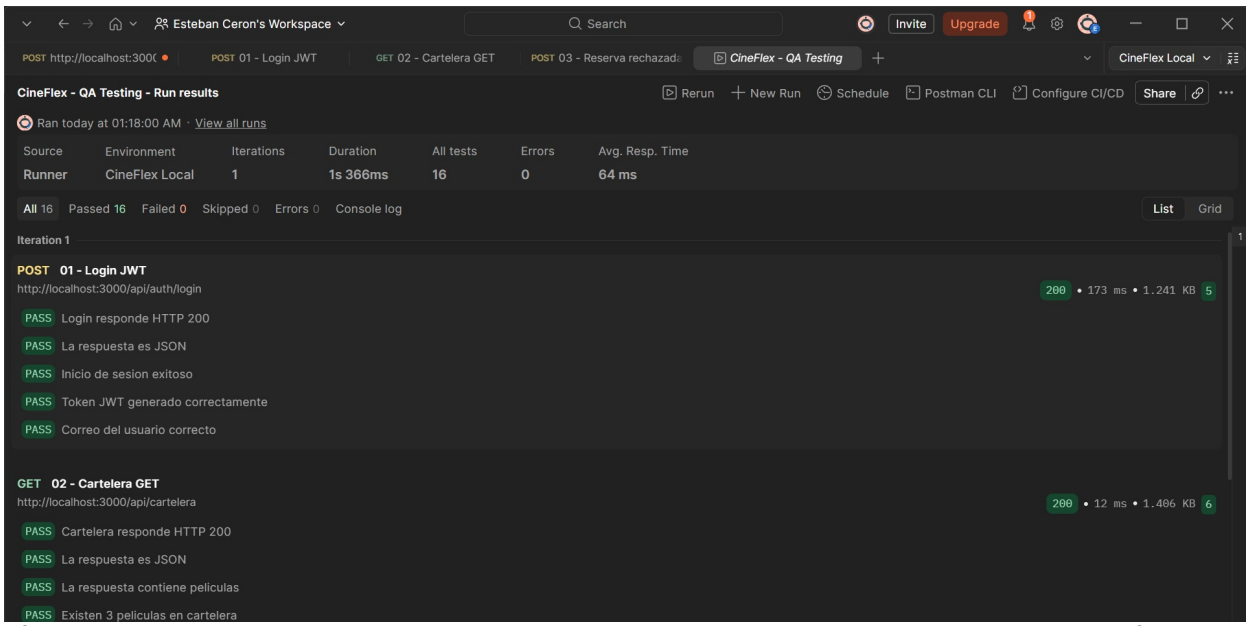


Qué hicimos: seleccionamos un pago aprobado en el simulador. **Dónde:** formulario del Frontend y servicio de reservas del Backend. **Qué comprueba:** se genera un identificador de ticket y la reserva queda «CONFIRMADO»; el asiento pasa a ocupado dentro del estado en memoria.

Figura B2. Ticket virtual emitido después de un pago de prueba autorizado.

ANEXO C. Pruebas API y automatización en GitHub

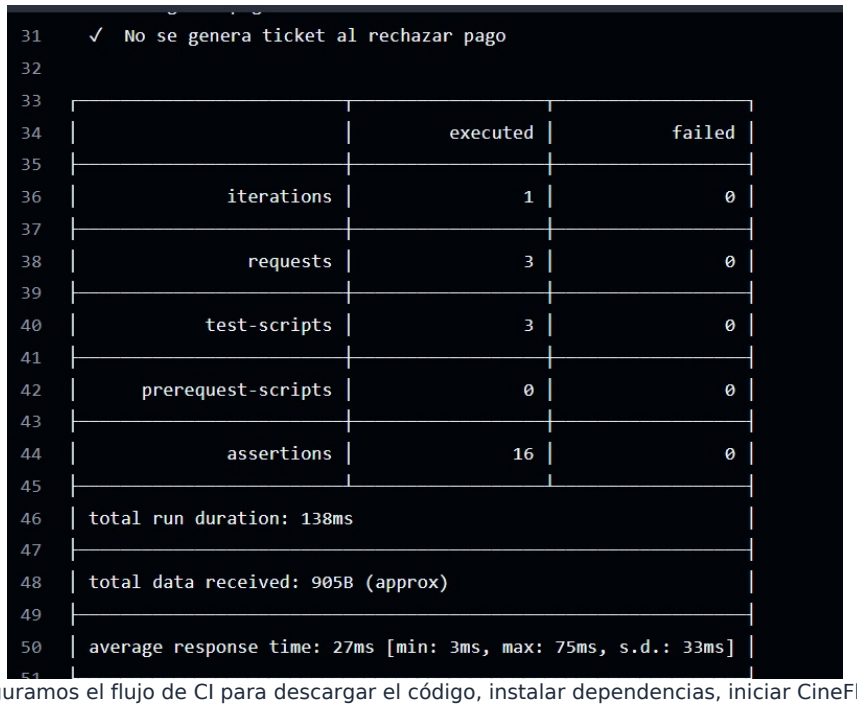
C.1 Postman: pruebas funcionales sobre tres endpoints



Qué hicimos: ejecutamos Login JWT (5 aserciones), cartelera autenticada (6) y reserva con tarjeta falsa (5). **Dónde:** api_tests/CineFlex_API_Collection.json y api_tests/CineFlex_Environment.json. **Resultado:** 16 aserciones aprobadas y ninguna fallida. La cartelera incluyó el umbral de respuesta inferior a 300 ms.

Figura C1. Runner de Postman con el resumen de 16 pruebas aprobadas.

C.2 GitHub Actions: ejecución automática con Newman



Qué hicimos: configuramos el flujo de CI para descargar el código, instalar dependencias, iniciar CineFlex y ejecutar la colección usando Newman al hacer push. **Dónde:** .github/workflows/qa-pipeline.yml. **Resultado:** workflow «pruebas-api» exitoso; 16/16 aserciones, 0 fallos.

Figura C2. Tabla real del paso «Ejecutar pruebas Newman» en GitHub Actions.

ANEXO D. Automatización de interfaz con Selenium/POM

La ejecución compartida en PowerShell reportó **15 pruebas, 15 aprobadas, 0 fallidas** y una duración total de **27,22 segundos**. Esta cifra proviene de los registros de consola; **no corresponde a una captura fotográfica recuperada**.

D.1 Separación de responsabilidades del patrón POM

Casos	Acción validada	Clase Page Object usada
1	Inicio de sesión correcto	LoginPage.js
2–5	Tres películas, títulos, selección y regreso	CarteleraPage.js
6–10	Horarios, 12 asientos, ocupado y validación sin asiento	ReservaPage.js
11–15	Selección, resumen, pago rechazado/aprobado y cambio de horario	ReservaPage.js

D.2 Dónde lo hicimos

Clases: ui_tests/pages/LoginPage.js, ui_tests/pages/CarteleraPage.js y ui_tests/pages/ReservaPage.js. Son responsables de los selectores y las acciones sobre Chrome, **sin aserciones dentro de las Pages**.

Suite: ui_tests/test_suite.js; allí están las pruebas y sus aserciones con node:test. **Comando:** desde ui_tests/, ejecutar npm test. Se utiliza selenium-webdriver para controlar Chrome.

D.3 Ejemplo visual de un escenario automatizado

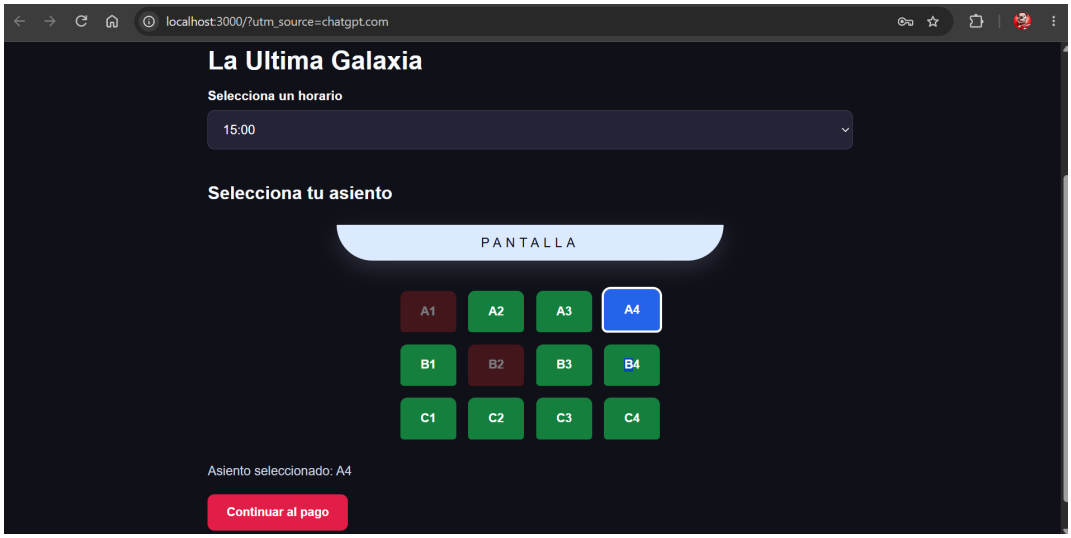


Figura D1. La interfaz de selección de asientos es objeto de varios casos automatizados: cantidad de asientos, no selección de ocupados, selección de libres y actualización de la etiqueta. Esta es una imagen del Frontend, **no una captura del resultado de Selenium**.

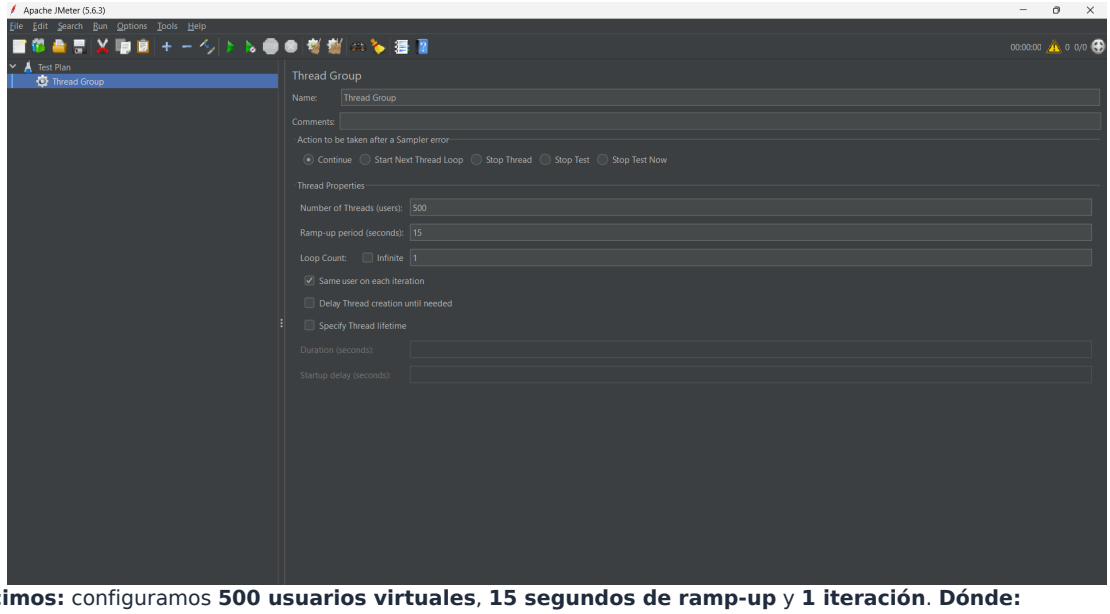
D.4 Casos adicionales exigidos en la rúbrica

> **Pendientes de verificación explícita**
>
> El enunciado exige además probar bloqueo tras **tres contraseñas erróneas**, presencia explícita del título **«Cartelera» en el DOM** y **cerrar sesión + botón Atrás** del navegador. No deben declararse aprobados hasta añadirlos y ejecutarlos; las 15 pruebas aprobadas existentes cubren otros escenarios.

ANEXO E. Diseño de la carga con JMeter

El experimento usa usuarios virtuales escalonados contra el servidor local. La autenticación se ejecuta una vez en un **setUp Thread Group** y las solicitudes de asientos emplean el JWT compartido.

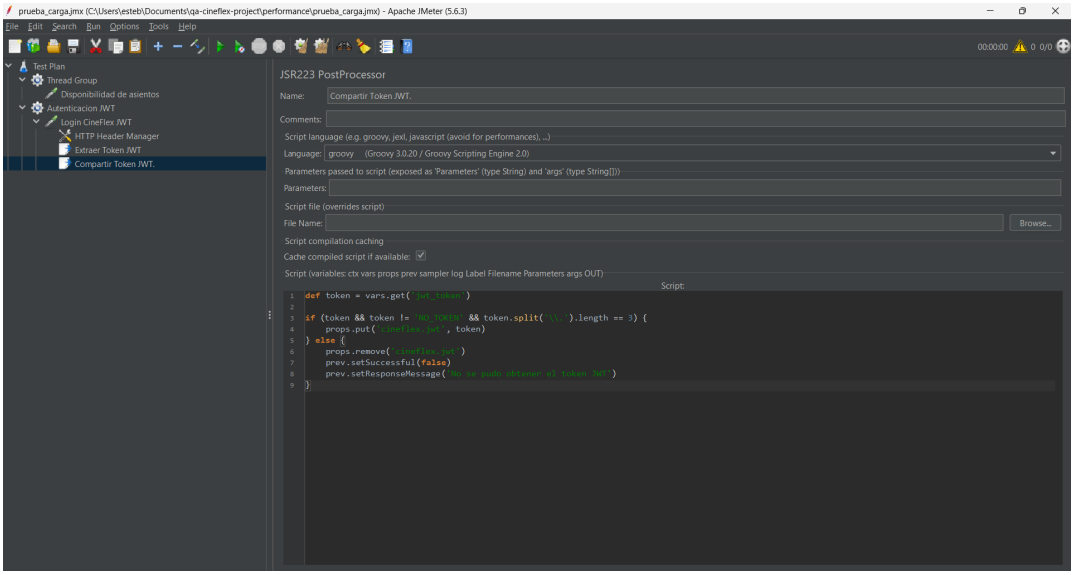
E.1 Configuración del Thread Group



Qué hicimos: configuramos **500 usuarios virtuales**, **15 segundos de ramp-up** y **1 iteración**. **Dónde:** performance/prueba_carga.jmx en Apache JMeter 5.6.3. **Ruta de carga:** GET /api/peliculas/1/asientos.

Figura E1. Parámetros de arranque gradual de los usuarios de prueba.

E.2 Extracción y reutilización del JWT



Qué hicimos: enviamos POST /api/auth/login, extrajimos $.token con JSON Extractor y lo compartimos entre grupos mediante JSR223/Groovy y el encabezado Authorization: Bearer. **Resultado:** la petición protegida respondió HTTP 200 en la prueba inicial.

Figura E2. PostProcessor Groovy que comparte el JWT de prueba con el grupo de carga.

ANEXO F. Resultados medidos de rendimiento

La ejecución final se realizó en **modo no gráfico** con JMeter, sin interferencia de su propia interfaz. El archivo de resultados se guardó localmente como performance/resultados_carga.jtl; por seguridad y tamaño se excluye de Git mediante .gitignore.

F.1 Summary Report de la ejecución

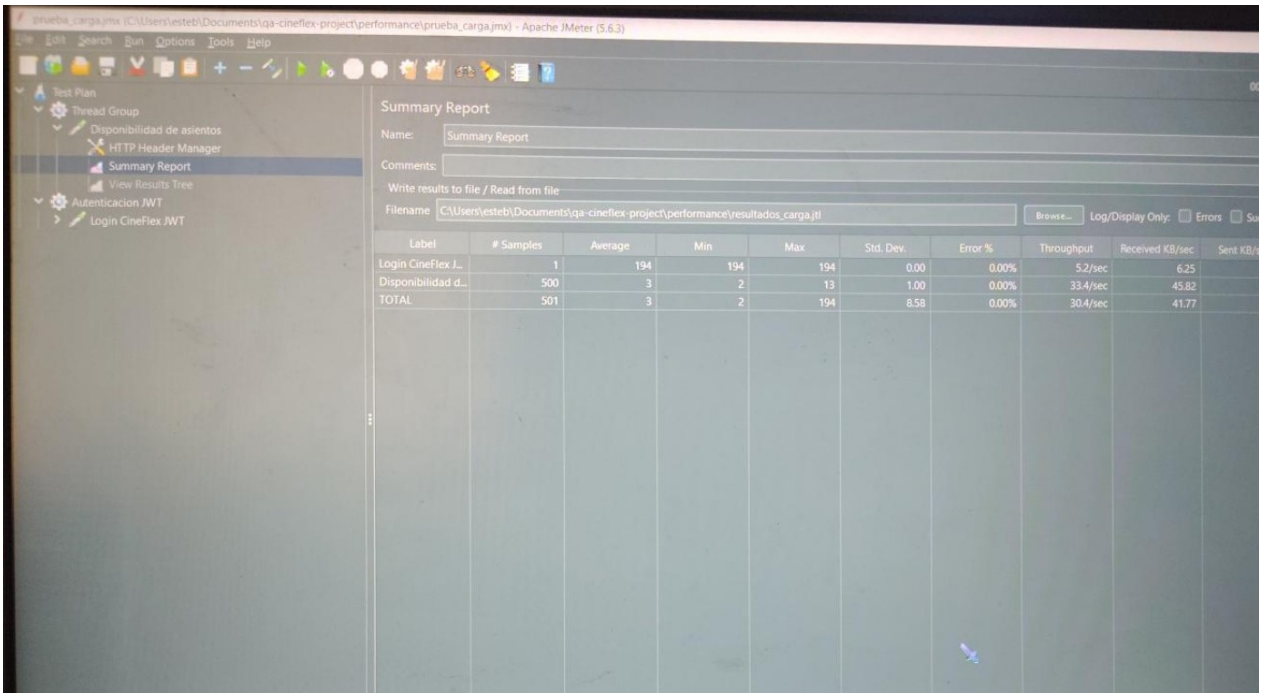


Figura F1. Recorte de la fotografía del Summary Report cargado desde el archivo JTL de la prueba final. Los datos de la siguiente tabla proceden de esa evidencia.

Indicador	Consulta de asientos	Total con login
Solicitudes	500	501
Tasa de error	0,00 %	0,00 %
Tiempo medio	3 ms	3 ms
Throughput	33,4 solicitudes/s	30,4 solicitudes/s

Interpretación correcta

Se iniciaron 500 usuarios virtuales **gradualmente durante 15 segundos**. Los resultados no prueban que existieran 500 peticiones simultáneas en un único instante ni certifican producción. Son métricas del servidor local y de una sola corrida.

ANEXO G. Rutas, evidencias y pendientes de entrega

Esta matriz permite localizar rápidamente cada trabajo en el repositorio y vincularlo con su evidencia. Los nombres de las capturas corresponden al paquete de evidencias recuperadas durante la actividad.

Componente o actividad	Archivo o carpeta	Evidencia visual
Servidor Express y JWT	backend/server.js	01_backend_health.png
Interfaz y selección de asiento	frontend/index.html; app.js; styles.css	03_asientos_seleccion.png
Stub bancario y tickets	POST /api/reservas (backend/server.js)	04_pago_rechazado.png; 05_ticket_confirmado.png
Colección de pruebas API	api_tests/*.json	06_postman_16_de_16.png
Pipeline CI con Newman	.github/workflows/qa-pipeline.yml	08_github_actions_16_de_16.png
Pruebas UI con POM	ui_tests/pages/; ui_tests/test_suite.js	Resultado de consola 15/15; captura pendiente
Carga de 500 usuarios	performance/prueba_carga.jmx	10_jmeter_500_usuarios_config.png
Reporte final de carga	performance/resultados_carga.jtl (local)	11_jmeter_summary_500_consultas.jpg

Pendientes para cerrar la entrega conforme a la rúbrica

1. Selenium: añadir y ejecutar los casos específicos de bloqueo tras tres fallos de login, título Cartelera en el DOM y logout seguido de navegación Atrás. Mantener separada la lógica de las Page Objects.

2. README: explicar instalación, comandos de ejecución, resultados y publicar una imagen clara del Summary Report dentro del README. Incluir al inicio el enlace al video final.

3. Video: mostrar arquitectura, Page Object Model y ejecución real de pruebas. El enunciado indica 3 minutos, pero se documentó autorización del profesor para 4 minutos; confirmar esa excepción al entregar.

4. Seguridad: evitar publicar archivos .env y tokens JWT; revisar alertas de dependencias de desarrollo y explicar la limitación de persistencia en memoria.

> **Alcance y procedencia**
>
> Las imágenes de este anexo son capturas reales compartidas durante el desarrollo de CineFlex; algunos recortes se hicieron únicamente para mejorar la lectura. Los resultados de Selenium sin imagen se describen desde la salida de PowerShell suministrada. No se han creado capturas falsas.

Repositorio: **https://github.com/pollofrito18/qa-cineflex-project**